

华东师范大学软件工程学院实验报告

姓名：高珩博

学号：10255101445

实验编号：Lab5

实验名称：MallocLab

一、实验目的

1. 实验需要补充完整 mm.c 文件，实现一个显式动态内存分配器的功能，尽可能达到正确、快速和高效。
2. 修改完成后执行 make clean 和 make 进行编译，运行 driver 自动评分程序进行评分，得分越高则代表动态内存分配器的正确性、空间利用率和吞吐量越好。

二、实验内容与实验步骤

1. 实验内容与总览

查看 mm.h 中声明的函数和 mm.c 中的内容，本实验需要实现一个完整的动态内存分配器，包含以下四个核心函数：

1. int mm_init(void)

在调用 mm_malloc、mm_realloc 或 mm_free 之前，应用程序（例如自动评分程序）会调用 mm_init，来执行任何必要的初始化，例如分配初始堆区域。返回值-1 表示执行初始化时出现问题，否则返回值为 0。

2. void *mm_malloc(size_t size)

mm_malloc 返回一个指针，指向荷载至少为 size 个字节的新分配块的荷载区。这个新分配的块应位于堆区域内，且不应与其它任何已分配的块重叠。

需要与标准 C 库（libc）中的 malloc 进行比较。由于 libc 的 malloc 总是返回指向 8 字节对齐的荷载区的指针，因此 mm_malloc 也应该总是返回 8 字节对齐的指针。

3. void mm_free(void *ptr)

mm_free 释放指针 ptr 指向的块，它的返回值为空。指针 ptr 是由之前调用 mm_malloc 或 mm_realloc 返回的且尚未释放的指针。

4. void *mm_realloc(void *ptr, size_t size)

mm_realloc 返回一个指向大小至少为 size 个字节的重新分配块的指针，具有以下约束：

约束 1：如果 ptr 等于 NULL，那么对此函数的调用等效于调用 mm_malloc(size)；

约束 2：如果 size 等于 0，那么对此函数的调用等效于调用 mm_free(ptr)；

约束 3：如果 ptr 不为空，则 ptr 必须是之前通过调用 mm_malloc 或 mm_realloc 返回的。对 mm_realloc 的调用会使得 ptr 指向的块的大小重新调整为 size 个字节，并返回新块的地址。请注意，新块的地址可能与旧块的地址相同，也可能不同，具体取决于实现方式、旧块中的内部碎片量以及参数 size 的大小。

约束 4：新块的内容与旧块的内容需要保持相同。例如，如果旧块是 8 个字节新块是 12 个字节，那么新块的前 8 个字节与旧块的前 8 个字节相同，新块的最后 4 个字节未初始化。类似的，如果旧块是 8 个字节新块是 4 个字节，那么新块的内容与旧块的前 4 个字节相同。

2. 编写函数

明细目标之后，分别编写四个函数

在此之前，基于所需堆的基本结构有以下宏定义与全局变量

```

/* 对齐 - 所有块大小均为8的倍数 */
#define ALIGNMENT 8
#define ALIGN(size) (((size) + (ALIGNMENT-1)) & ~0x7)

/* 基本常量 */
#define WSIZE 4 /* 字大小(字节) */
#define DSIZE 8 /* 双字大小 */
#define CHUNKSIZE (1<<7) /* 堆扩展程度: 128字节(原为4096) */

#define MAX(x, y) ((x) > (y) ? (x) : (y))

/*
 * 将大小、分配位(bit 0)、前块分配位(bit 1)打包成一个字
 * alloc = 0 表示空闲, 1 表示已分配
 * prev_alloc = 0 表示前一个块空闲, 1 表示已分配
 */
#define PACK(size, alloc, palloc) ((size) | (alloc) | ((palloc) << 1))

/* 读取和写入地址p处的字 */
#define GET(p) (*(unsigned int *) (p))
#define PUT(p, val) (*(unsigned int *) (p) = (val))

/* 从头部/尾部字中提取大小、分配位和前块分配位 */
#define GET_SIZE(p) (GET(p) & ~0x7)
#define GET_ALLOC(p) (GET(p) & 0x1)
#define GET_PREV_ALLOC(p) ((GET(p) >> 1) & 0x1)

/* 根据块指针bp计算头部和尾部地址 */
#define HDRP(bp) ((char *) (bp) - WSIZE)
#define FTRP(bp) ((char *) (bp) + GET_SIZE(HDRP(bp)) - DSIZE)

/* 根据块指针bp计算下一个/前一个块的地址 */
#define NEXT_BLKP(bp) ((char *) (bp) + GET_SIZE((char *) (bp) - WSIZE))
#define PREV_BLKP(bp) ((char *) (bp) - GET_SIZE((char *) (bp) - DSIZE))

/* 显式空闲链表: 前驱和后继指针存储在空闲块的有效载荷区 */
#define GET_PRED(bp) (*(void **) (bp))
#define GET_SUCC(bp) (*(void **) ((char *) (bp) + WSIZE))
#define SET_PTR(p, val) (*(void **) (p) = (val))

/* 分离空闲链表参数 */
#define LIST_SIZE 20 /* 大小类的数量 */
#define MIN_BLOCK 16 /* 最小块大小: 头部(4)+前驱(4)+后继(4)+尾部(4) = 16 */
#define SPLIT_MIN (2*DSIZE) /* 最小分割剩余: 16字节 - 最小有效空闲块大小 */

/* 全局变量 */

/* 全局变量 */
static char *heap_listp = NULL; /* 指向第一个块(序言块有效载荷) */
static void *free_list[LIST_SIZE]; /* 分离空闲链表头指针数组 */

/* 前向声明 */
static void *extend_heap(size_t words);
static void *best_fit(size_t asize);
static void place(void *bp, size_t asize);
static void *coalesce(void *bp);
static void add_to_free_list(void *bp);
static void remove_from_free_list(void *bp);
static int get_index(size_t size);
static void init_free_list(void);

```

1. 辅助函数:

为了使后续代码更加简洁, 加入辅助函数。

```

/*
 * init_free_list - 初始化所有分离空闲链表头为NULL。
 */
static void init_free_list(void)
{
    int i;
    for (i = 0; i < LIST_SIZE; i++)
        free_list[i] = NULL;
}

static int get_index(size_t size)
{
    if (size <= 16) return 0;
    if (size <= 24) return 1;
    if (size <= 32) return 2;
    if (size <= 48) return 3;
    if (size <= 64) return 4;
    if (size <= 96) return 5;
    if (size <= 128) return 6;

    /* 大小 > 128 时使用2的幂分桶 */
    int index = 7;
    size = size / 128;
    while (size > 1 && index < LIST_SIZE - 1) {
        size >>= 1;
        index++;
    }
    return index;
}

```

用于实现空闲链表初始化与大小类映射

思路: 将空闲块按大小分为 20 个类别, 不超过 128 字节时采用细粒度划分——针对 16/24/32/48/64/96/128 分别设置独立类别, 因为实验中大部分分配请求集中在小尺寸范围, 细粒度分类可以显著减少类内搜索时间。大于 128 字节时采用 2 的幂分桶策略。`get\_index` 函数将任意块大小映射到对应的类别索引。

```

static void add_to_free_list(void *bp)
{
    size_t size = GET_SIZE(HDRP(bp));
    int index = get_index(size);

    /* 插入链表头部: bp->pred = NULL, bp->succ = 原链表头 */
    SET_PTR(bp, NULL);
    SET_PTR((char *)bp + WSIZE, free_list[index]);
    if (free_list[index] != NULL)
        SET_PTR(free_list[index], bp);
    free_list[index] = bp;
}

/*
 * remove_from_free_list - 从分离空闲链表中移除空闲块。
 * 该块当前必须在某个空闲链表中。
 */
static void remove_from_free_list(void *bp)
{
    size_t size = GET_SIZE(HDRP(bp));
    int index = get_index(size);

    /* 更新前驱的后继, 若bp是链表头则更新链表头 */
    if (GET_PRED(bp) != NULL)
        SET_PTR((char *)GET_PRED(bp) + WSIZE, GET_SUCC(bp));
    else
        free_list[index] = GET_SUCC(bp);

    /* 更新后继的前驱 */
    if (GET_SUCC(bp) != NULL)
        SET_PTR(GET_SUCC(bp), GET_PRED(bp));
}

```

用于操作显式空闲链表

```

static void *extend_heap(size_t words)
{
    char *bp;
    size_t size;

    /* 保持对齐: 确保字数为偶数 */
    size = (words % 2) ? (words + 1) * WSIZE : words * WSIZE;
    if ((long)(bp = mem_sbrk(size)) == -1)
        return NULL;

    /* 从当前结尾块头部读取其prev_alloc */
    int palloc = GET_PREV_ALLOC(HDRP(bp));

    /* 初始化新空闲块的头部和脚部 */
    PUT(HDRP(bp), PACK(size, 0, palloc));
    PUT(FTRP(bp), PACK(size, 0, palloc));

    /* 创建新的结尾块: alloc=1, palloc=0 (新块在其之前且空闲) */
    PUT(HDRP(NEXT_BLKP(bp)), PACK(0, 1, 0));

    /* 尝试与前块合并(也会将结果加入空闲链表) */
    return coalesce(bp);
}

```

用于扩展堆空间

```
static void *best_fit(size_t asize)
{
    int i;
    void *bp;
    void *best = NULL;
    size_t best_size = 0;

    /* 从匹配的大小类开始向上搜索 */
    for (i = get_index(asize); i < LIST_SIZE; i++) {
        for (bp = free_list[i]; bp != NULL; bp = GET_SUCC(bp)) {
            size_t curr_size = GET_SIZE(HDRP(bp));
            if (curr_size >= asize) {
                if (best == NULL || curr_size < best_size) {
                    best = bp;
                    best_size = curr_size;
                    /* 正好匹配 - 不可能有更好的, 立即返回 */
                    if (curr_size == asize)
                        return best;
                }
            }
        }
    }
    return best;
}
```

该函数最佳适配策略在分离空闲链表中搜索

思路: 从匹配的大小类`get\_index(asize)`开始向上遍历, 检查每个块是否空闲且足够大, 记录最小的满足条件的块。与首次适配相比, 最佳适配能最小化外部碎片, 选择刚好够用的最小块, 使得分割后的剩余空间尽可能大, 更容易被后续请求复用。一旦找到正好等于请求大小的块, 立即返回

```
static void place(void *bp, size_t asize)
{
    size_t csize = GET_SIZE(HDRP(bp));
    size_t remainder = csize - asize;

    if (remainder >= SPLIT_MIN) {
        /* 分割: 前部分分配(无脚部), 剩余部分空闲(有脚部) */
        alloc_hdr(bp, asize);
        free_blk(NEXT_BLK(bp), remainder, 1); /* palloc=1: 前块已分配 */
        coalesce(NEXT_BLK(bp)); /* 合并剩余块(加入空闲链表) */
    } else {
        /* 不分割 - 整个块分配出去 */
        alloc_hdr(bp, csize);
    }
}
```

该函数用来在找到的空闲块中放置请求的块

```

static void *coalesce(void *bp)
{
    size_t prev_alloc = GET_PREV_ALLOC(HDRP(bp));
    size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLK(bp)));
    size_t size = GET_SIZE(HDRP(bp));
    size_t final_palloc = prev_alloc;

    /* 从空闲链表中移除相邻的空闲块 */
    if (!prev_alloc) {
        final_palloc = GET_PREV_ALLOC(HDRP(PREV_BLK(bp)));
        remove_from_free_list(PREV_BLK(bp));
    }
    if (!next_alloc)
        remove_from_free_list(NEXT_BLK(bp));

    /* 根据四种情况合并 */
    if (prev_alloc && next_alloc) {
        /* 情况1: 前后都已分配 - 不合并 */
    }
    else if (prev_alloc && !next_alloc) {
        /* 情况2: 后块空闲 - 与后块合并 */
        size += GET_SIZE(HDRP(NEXT_BLK(bp)));
    }
    else if (!prev_alloc && next_alloc) {
        /* 情况3: 前块空闲 - 与前块合并 */
        size += GET_SIZE(HDRP(PREV_BLK(bp)));
        bp = PREV_BLK(bp);
    }
    else {
        /* 情况4: 前后都空闲 - 与前后块合并 */
        size += GET_SIZE(HDRP(PREV_BLK(bp))) +
            GET_SIZE(HDRP(NEXT_BLK(bp)));
        bp = PREV_BLK(bp);
    }

    /* 写入合并后的空闲块(头部+脚部)并加入空闲链表 */
    PUT(HDRP(bp), PACK(size, 0, final_palloc));
    PUT(FTRP(bp), PACK(size, 0, final_palloc));
    {
        unsigned int *nh = (unsigned int *)HDRP(NEXT_BLK(bp));
        *nh = (*nh & ~0x2) | 0x0;
    }
    add_to_free_list(bp);
    return bp;
}

```

该函数用于合并空闲块，传入参数 `bp` 是当前块的指针，返回合并后块的指针。通过直接读取当前块头部的 `bit 1` (`GET_PREV_ALLOC`) 实现，这样做的好处是：即使前一个块是已分配的，也能正确判断其状态。合并前需要先将相邻空闲块从各自的空闲链表中移除，合并完成后将结果块加入适当的空闲链表。同时更新合并块之后那个块的 `prev_alloc` 位为 0。处理四种情况：

1. 前后块都已分配 — 不合并，直接返回
2. 前块已分配，后块空闲 — 与后块合并，更新当前块的头部和脚部
3. 前块空闲，后块已分配 — 与前块合并，更新前块头部和当前块脚部，指针前移
4. 前后块都空闲 — 与前后块合并，更新前块头部和后块脚部，指针前移

```

static void alloc_hdr(void *bp, size_t size)
{
    int pa = GET_PREV_ALLOC(HDRP(bp));
    PUT(HDRP(bp), PACK(size, 1, pa));
    /* 下一个块的prev_alloc = 1 (本块已分配) */
    unsigned int *nh = (unsigned int *)HDRP(NEXT_BLK(bp));
    *nh = (*nh & ~0x2) | 0x2;
}

②~
static void alloc_hdr_explicit(void *bp, size_t size, int palloc)
{
    PUT(HDRP(bp), PACK(size, 1, palloc));
    unsigned int *nh = (unsigned int *)HDRP(NEXT_BLK(bp));
    *nh = (*nh & ~0x2) | 0x2;
}

②~
static void free_blk(void *bp, size_t size, int palloc)
{
    PUT(HDRP(bp), PACK(size, 0, palloc));
    PUT(FTRP(bp), PACK(size, 0, palloc));
    unsigned int *nh = (unsigned int *)HDRP(NEXT_BLK(bp));
    *nh = (*nh & ~0x2) | 0x0;
}

/*

```

这三个辅助函数用于块头部/脚部写入

每次写入块头部/脚部时，都必须同步下一个块头部中的 `prev_alloc` 位，`'alloc_hdr'` 从块的旧头部读取 `prev_alloc`，而 `'alloc_hdr_explicit'` 用于新分割出的块。`'free_blk'` 同时写入头部和脚部，确保后续的 `'PREV_BLKp'` 能够正确计算前块地址。

2. `int mm_init(void):`

```
/*
 * mm_init - 初始化内存分配器。
 * 创建初始堆结构(序言块+结尾块)，并扩展堆获得第一个空闲块。
 * 实现思路：
 *   1. 申请4字(16字节)的初始堆空间：
 *       第0字：对齐填充
 *       第1-2字：序言块(8字节，已分配)
 *       第3字：结尾块(0字节，已分配)
 *   2. 初始化堆指针指向序言块之后
 *   3. 扩展堆空间获得初始空闲块
 */
int mm_init(void)
{
    /* 初始化分离空闲链表 */
    init_free_list();

    /* 申请4个字的初始堆空间 */
    if ((heap_listp = mem_sbrk(4 * WSIZE)) == (void *)-1)
        return -1;

    /* 堆初始化布局：
     * [对齐填充:4字节] - 字0
     * [序言块头部:4字节, PACK(DSIZE,1,1)] - 字1 (前块=填充=已分配)
     * [序言块脚部:4字节, PACK(DSIZE,1,1)] - 字2
     * [结尾块头部:4字节, PACK(0,1,1)] - 字3 (前块=序言=已分配)
     */
    PUT(heap_listp, 0);
    PUT(heap_listp + (1 * WSIZE), PACK(DSIZE, 1, 1));
    PUT(heap_listp + (2 * WSIZE), PACK(DSIZE, 1, 1));
    PUT(heap_listp + (3 * WSIZE), PACK(0, 1, 1));
    heap_listp += (2 * WSIZE); /* 堆指针指向序言块之后(第一个有效块位置) */

    /* 扩展堆获得初始空闲块 */
    if (extend_heap(CHUNKSIZE / WSIZE) == NULL)
        return -1;

    return 0;
}
```

实现思路如图中注释

3. `void *mm_malloc(size_t size):`

```
void *mm_malloc(size_t size)
{
    size_t asize; /* 调整后的块大小 */
    size_t extendsize; /* 需要扩展的堆大小 */
    char *bp;

    /* 忽略零字节请求 */
    if (size == 0)
        return NULL;

    /* 调整块大小：
     * 已分配块仅头部开销(WSIZE)，无脚部(Footer消除)。
     * 最小块为2*DSIZE=16字节(必须能容纳空闲块：头部+前驱+后继+脚部)。
     */
    asize = ALIGN(size + WSIZE);
    if (asize < 2 * DSIZE)
        asize = 2 * DSIZE;

    /* 在分离空闲链表中搜索合适块 */
    if ((bp = best_fit(asize)) != NULL) {
        remove_from_free_list(bp);
        place(bp, asize);
        return bp;
    }

    /* 没有找到合适块 - 扩展堆 */
    extendsize = MAX(asize, CHUNKSIZE);
    if ((bp = extend_heap(extendsize / WSIZE)) == NULL)
        return NULL;
    /* extend_heap返回已在空闲链表中的块 - 先移除再放置 */
    remove_from_free_list(bp);
    place(bp, asize);
    return bp;
}
```

思路如下：

1. 处理零字节请求的特殊情况(直接返回 `NULL`)。
2. 调整请求大小:由于辅助函数去掉了脚部，已分配块仅需要 4 字节头部开销，而非传统的 8 字节。加上对齐到 8 字节的要求，以及最小块 16 字节的限制。

3. 调用 `best_fit` 在分离空闲链表中搜索合适块, 从匹配的大小类开始, 向上遍历所有更大的类, 找到最小的满足条件的块。
4. 如果找到合适块: 从空闲链表移除, 调用 `place` 放置请求块(剩余空间大于等于 16 字节时分割)。
5. 如果没找到合适块: 扩展堆空间(至少扩展 `CHUNKSIZE=128` 字节), 在新扩展的空间中放置请求块。注意 `extend_heap` 返回的块已在空闲链表中, 需要先移除。

4. `void mm_free(void *bp):`

```
/*
 * mm_free - 释放之前由mm_malloc或mm_realloc分配的块。
 * 标记块为空闲(更新头部和脚部), 然后尝试与相邻空闲块合并。
 * 释放时需要写入脚部(空闲块始终有脚部)。
 */
void mm_free(void *bp)
{
    if (bp == NULL)
        return;

    size_t size = GET_SIZE(HDRP(bp));

    /* 空闲块需要头部+脚部。palloc从现有头部读取。 */
    int palloc = GET_PREV_ALLOC(HDRP(bp));
    PUT(HDRP(bp), PACK(size, 0, palloc));
    PUT(FTRP(bp), PACK(size, 0, palloc));
    {
        unsigned int *nh = (unsigned int *)HDRP(NEXT_BLKP(bp));
        *nh = (*nh & ~0x2) | 0x0;
    }

    /* 与相邻空闲块合并(包含空闲链表操作) */
    coalesce(bp);
}
```

该函数传入由`mm\_malloc`返回的有效指针作为要释放的块指针。实现思路为: 先获取块大小; 标记块为空闲, 由于已分配块原本没有脚部, 释放时必须补写脚部以便后续的`PREV\_BLK`操作; 保留头部中已有的`prev\_alloc`信息; 更新下一个块头部的`prev\_alloc`位为0(表示本块现在空闲); 最后调用`coalesce`与相邻的空闲块合并, 合并后的结果块会被`coalesce`自动加入合适的空闲链表。

5. `void *mm_realloc(void *ptr, size_t size):`

```
void *mm_realloc(void *ptr, size_t size) {
    if (ptr == NULL) return mm_malloc(size); // 相当于 malloc
    if (size == 0) { mm_free(ptr); return NULL; } // 相当于 free

    // 计算调整后的新大小(已分配块仅 WSIZE 头部开销)
    size_t asize = ALIGN(size + WSIZE);
    if (asize < 2 * DSIZ) asize = 2 * DSIZ;

    size_t oldsize = GET_SIZE(HDRP(ptr));
    if (oldsize == asize) return ptr; // 大小不变, 无需操作

    // --- 情况 1: 缩小 ---
    if (oldsize > asize) {
        alloc_hdr(ptr, asize); // 缩小后的分配块
        free_blk(NEXT_BLKP(ptr), oldsize - asize, 1); // 剩余部分标记为空闲
        coalesce(NEXT_BLKP(ptr)); // 合并剩余块
        return ptr;
    }

    // --- 情况 2: 多块向前吸收 ---
    // 统计 ptr 之后连续的空闲块, 如果累计空间足够则全部吸收
    void *next = NEXT_BLKP(ptr);
    size_t next_alloc = GET_ALLOC(HDRP(next));

    if (!next_alloc) {
```

```

size_t total = oldsize;
void *cursor = next;
while (!GET_ALLOC(HDRP(cursor)) && GET_SIZE(HDRP(cursor)) > 0) {
    total += GET_SIZE(HDRP(cursor));
    if (total >= asize) break;
    cursor = NEXT_BLKPTR(cursor);
}
if (total >= asize) {
    // 移除从 next 到 cursor 的所有空闲块
    void *rm = next;
    while (1) {
        remove_from_free_list(rm);
        if (rm == cursor) break;
        rm = NEXT_BLKPTR(rm);
    }
    // 分配 asize, 剩余部分释放
    size_t remainder = total - asize;
    if (remainder >= SPLIT_MIN) {
        alloc_hdr(ptr, asize);
        free_blk(NEXT_BLKPTR(ptr), remainder, 1);
        coalesce(NEXT_BLKPTR(ptr));
    } else {
        alloc_hdr(ptr, total); // 剩余太小, 全部吸收
    }
    return ptr;
}
}

// --- 情况 3: 向前扩展(与前块合并, 用 memmove 迁移数据) ---
size_t prev_alloc = GET_PREV_ALLOC(HDRP(ptr));
if (!prev_alloc) {
    void *prev = PREV_BLKPTR(ptr);
    size_t prev_size = GET_SIZE(HDRP(prev));
    size_t combined = oldsize + prev_size;
    if (!next_alloc) combined += GET_SIZE(HDRP(next));

    if (combined >= asize) {
        remove_from_free_list(prev);
        if (!next_alloc) remove_from_free_list(next);

        // 将数据向前迁移到合并块的起始位置
        memmove(prev, ptr, oldsize - WSIZE);

        // 分配 asize, 处理剩余空间
        size_t total = oldsize + prev_size;
        if (!next_alloc) total += GET_SIZE(HDRP(next));
        int prev_palloc = GET_PREV_ALLOC(HDRP(prev));

        if (total - asize >= SPLIT_MIN) {
            alloc_hdr_explicit(prev, asize, prev_palloc);
            free_blk(NEXT_BLKPTR(prev), total - asize, 1);
            coalesce(NEXT_BLKPTR(prev));
        } else {
            alloc_hdr_explicit(prev, total, prev_palloc);
        }
        return prev;
    }
}

// --- 情况 4: 堆尾扩展(最后一个块直接扩展堆) ---
{
    void *check = NEXT_BLKPTR(ptr);
    if (GET_SIZE(HDRP(check)) == 0) { // 后块是结尾块(size=0)
        size_t need = asize - oldsize;
        size_t ext_words = MAX(need, CHUNKSIZE) / WSIZE;
        char *nb = extend_heap(ext_words);
    }
}

```



```

if (nb != NULL) {
    // 吸收新扩展的空间
    void *absorb = NEXT_BLKPTR(ptr);
    remove_from_free_list(absorb);
    size_t abs_size = GET_SIZE(HDRP(absorb));
    size_t total = oldsize + abs_size;
    // 分配 asize, 处理剩余
    if (total - asize >= SPLIT_MIN) {
        alloc_hdr(ptr, asize);
        free_blk(NEXT_BLKPTR(ptr), total - asize, 1);
        coalesce(NEXT_BLKPTR(ptr));
    } else {
        alloc_hdr(ptr, total);
    }
    return ptr;
}

// --- 情况 5: 回退 --- malloc 新块 + 复制数据 + 释放旧块 ---
void *newptr = mm_malloc(size);
if (newptr == NULL) return NULL;

size_t copySize = oldsize - WSIZE; // 有效载荷 = 块大小 - 头部(无脚部)
if (size < copySize) copySize = size;
memcpy(newptr, ptr, copySize);
mm_free(ptr);
return newptr;
}

```

思路, 针对五种典型场景分别采用最优策略:

情况 1 (缩小): 如果新大小小于旧大小, 原地缩小——将块的头部改为新大小, 剩余空间标记为空闲块, 调用 `coalesce` 尝试与后块合并。

情况 2 (多块向前吸收): 当前块空闲但单个后块空间不够时, 继续向后检查, 循环统计连续的空闲块数量, 如果累计空间足够则全部吸收。先统计后移除的策略避免了不足时需回退的复杂逻辑。

情况 3 (向前扩展): 如果前块空闲且合并后空间足够, 将前块从空闲链表移除, 用 `memmove` 将当前块的数据向前迁移到合并块的起始位置, 然后更新头部/脚部。如果后块也空闲可一并吸收。

情况 4 (堆尾扩展): 如果当前块是堆上最后一个块, 直接扩展堆——新空间紧接在当前块之后, 吸收新空间即可完成原地增长, 避免 `malloc`+复制+释放的开销。

情况 5 (回退): 以上优化均不适用时, 回退到标准实现: `mm_malloc` 新块 → `memcpy` 复制数据 → `mm_free` 旧块。

三、实验过程与分析

编写以上函数完成后, 用 `./mdriver -g -t ../traces/traces` 指令进行跑分

以下是跑分截图:

```

boyu@LAPTOP-0ABVIRCU:~/lab5/malloclab-handout$ ./mdriver -av -f short1-bal.rep
Results for mm malloc:
trace valid util ops secs Kops
0 yes 66% 12 0.000000 40000
Total 66% 12 0.000000 40000

Perf index = 39 (util) + 40 (thru) = 79/100
boyu@LAPTOP-0ABVIRCU:~/lab5/malloclab-handout$ ./mdriver -av -f short2-bal.rep
Measuring performance with gettimeofday().

Results for mm malloc:
trace valid util ops secs Kops
0 yes 99% 12 0.000000 40000
Total 99% 12 0.000000 40000

Perf index = 60 (util) + 40 (thru) = 100/100
boyu@LAPTOP-0ABVIRCU:~/lab5/malloclab-handout$ ./mdriver -av -t /home/boyu/lab5/traces/
traces
Using default tracefiles in /home/boyu/lab5/traces/traces/
Measuring performance with gettimeofday().

Results for mm malloc:
trace valid util ops secs Kops
0 yes 99% 5694 0.000176 32371
1 yes 100% 5848 0.000301 19409
2 yes 99% 6648 0.000231 28729
3 yes 100% 5380 0.000182 29496
4 yes 98% 14400 0.000264 54545
5 yes 96% 4800 0.000098 6879
6 yes 94% 4800 0.000462 10392
7 yes 55% 12000 0.000211 56872
8 yes 51% 24000 0.000512 46848
9 yes 48% 14401 0.000332 43363
10 yes 74% 14401 0.000421 61415
Total 83% 112372 0.003512 31996

Perf index = 50 (util) + 40 (thru) = 90/100

```

四、实验结果总结

最开始版本的代码，我自己的思路是使用隐式空闲链表，并在搜索最佳适配后立即合并。

然后发现跑分只有 72 分，询问 ai 问题发现，原因出在了隐式空闲链表在分配时需要线性扫描整个堆，同时最佳适配虽然能减少外部碎片，但在频繁分配/释放模式下仍会产生大量不可用的小碎片

于是借助 ai 进行优化，采用了显式空闲链表，并察觉到已分配块不需要脚部——脚部仅在合并时需要判断前一个块是否空闲，所以又对块读写入函数进行了修改。

最终优化到 90 分。

总体来说，实验非常难，很多时候我都是借助 ai 才能对具体内容有一定理解，从而进行代码的编写。内存分配器的工作模式有很多，还好在实验过程中多多少少记住了一些，之后还需要多次复习，总的来说，通过这次实验也让我对内存的管理也有了一些结合个人理解的认识。

自此，实验完成。

五、附录（源代码）

源码见报告正文部分